GÖDEL TRADER: SEPARATING GENUINE SELF-IMPROVEMENT FROM EVALUATOR EXPLOITATION IN TRADING-STRATEGY RESEARCH

Mohammad Shoaib Lari
Stanford University
shoaiblari@gmail.com
CS 329Z Midway Report, November 6, 2026
Code: https://github.com/shoaiblari/godel_trader
Video: https://youtu.be/YO-fM8ethbI

1 RESEARCH QUESTIONS

Quantitative strategy research is a loop: hypothesise a signal, implement it, backtest it, read the result, decide what to try next. LLM agents such as R&D-Agent(Q) (Li et al., 2025) automate the loop but keep its structure fixed. Gödel Agent (Yin et al., 2025) removes that restriction: the agent can read and rewrite every part of its own loop, including the code that does the rewriting. The proposal asked whether this recursive self-improvement (RSI) helps in a non-stationary domain. Our first experiments sharpened the question. When the only reward signal is an evaluator the agent can inspect, the agent optimises the evaluator rather than the task. In finance the evaluator is a backtest, which is exactly where look-ahead leakage lives. We therefore ask:

1. **RQ1 (capability).** Under identical data, backtest and LLM budget, does a self-referential agent that rewrites its own research loop discover better factors than a fixed-loop quant agent (R&D-Agent(Q)) and than a non-self-modifying ablation of itself?
2. **RQ2 (measurement).** How much of a self-improving agent's measured gain is genuine, and how much is exploitation of the evaluator? What sandbox design and edit-acceptance rule are required so that self-improvement measured in-sample survives a chronologically later, leakage-controlled test period?
3. **RQ3 (process).** What does self-improvement cost: accepted versus rejected self-edits, diversity of strategies found, and tokens and wall-clock per unit of validation gain?

The teachable outcome we are aiming for is not "Gödel Agent works for trading too" but a characterisation of when self-modification produces real improvement and what the evaluator must look like for that to be true.

2 ENVIRONMENT & DATA

2.1 CURRENT SUBSTRATE: GÖDEL AGENT ON MGSM

Gödel Agent ships with four benchmark tasks; we use MGSM (Shi et al., 2022), grade-school math word problems translated into eleven languages, 250 problems each, 2,750 in total. The upstream evaluation protocol, which we keep unchanged, shuffles the data with a fixed seed and holds out the first 128 problems as a validation pool and the next 800 as a test set. Each evaluation samples 50 validation problems afresh, scores exact match on the integer answer, and runs the 800-problem test set only if validation accuracy reaches 0.94, reporting a bootstrap confidence interval. The planner model is `o3-2025-04-16`; the solver calls `gpt-4o-mini-2024-07-18`. One run is capped at 100 planning steps and took 5–15 minutes and a few US dollars in our experience.

This substrate is not the target domain. It is the smallest environment in which the self-modification machinery runs end to end, and it already exposes the evaluator problem that shapes the trading environment below.

2.2 Planned trading environment

Data. Daily US equity prices and volumes from Yahoo Finance via `yfinance`, converted to Qlib's binary format (Yang et al., 2020) so that both our agent and the R&D-Agent(Q) baseline run on the same backtest engine. R&D-Agent's default data (CSI300, 2008–2020) is unsuitable: every current LLM has seen that period, and it cannot be extended past 2020. The universe will be a fixed list of liquid large-cap US stocks; we will state the survivorship caveat and, if time permits, use point-in-time index membership.

Splits and leakage control. Train 2010–2019, validation 2020–2023, test from the planner LLM's training cutoff onward, so that memorised market history cannot leak into the test period (Glasserman & Lin, 2023). Features are computed point-in-time only. Crucially, and this is the lesson of Section 4, *the labelled data and the scoring code will live in a separate evaluator process that the agent cannot import, read or modify*; the agent sees only a submit-and-score interface and summary feedback.

Metrics. For a candidate factor set: the information coefficient (IC, the cross-sectional correlation between the factor's predicted and realised next-period returns) and its rank version (rank IC), annualised excess return, information ratio, maximum drawdown and turnover from Qlib's top-k long-only backtest with transaction costs. Because the agent tests many candidates, we report the deflated Sharpe ratio (Bailey & López de Prado, 2014) on the test window.

Success criteria. RQ1: validation and test utility of the final agent versus the frozen R&D-Agent(Q) `fin_factor` loop and the non-self-modifying ablation, at matched LLM budget. RQ2: the gap between sandbox score and held-out test score, and an audit of every accepted self-edit classified as task improvement, evaluator exploitation, or neutral. RQ3: counts and costs as listed above.

3 Methods

3.1 The self-referential loop as implemented

Gödel Agent is a single Python process. Each `evolve` step sends the planner a goal prompt, a textual dump of the live Python namespace, and the recent action history, and requires it to reply with tool calls. Seven tools are exposed: display an analysis; list the environment; read any function's source; *replace any function or method in the running interpreter* (`action_adjust_logic`, implemented as `exec` of the new source followed by `setattr` on the module or class); *run arbitrary Python in the same interpreter* (`action_run_code`, whose new variables are merged into the module's globals); call an LLM; and evaluate the current `solver`. Executing the tools ends by calling `evolve` again, so the loop is mutual recursion capped at 100 steps. Modifications are in memory only; code on disk never changes, and the best agent found is serialised to a folder when the test score improves. The object being optimised is one function, `solver(agent, task)` $\rightarrow$ `answer`.

3.2 Local changes

Two patches, documented in the repository. (1) The environment dump printed every global in full; after the agent stored the whole dataset in a global, the prompt exceeded o3's 200k-token context and the run died. Each value is now truncated to 300 characters plus its type and length. (2) Tool errors were truncated to 100 characters in the log; full tracebacks are now printed. The agent's prompts, tools and evaluation protocol are untouched.

Run	Honest solver rewrites (validation acc.)	After lookup rewrite	Ended by
1	none evaluated	–	context overflow at step 19
2	0.78, 0.86, 0.86, 0.84 (4 rewrites)	1.00 valid / 1.00 test	100-step cap
3	0.88, 0.78, 0.82 (1 rewrite)	1.00 valid / 1.00 test	100-step cap

Table 1: The three runs. Honest rewrites were self-consistency voting over gpt-4o-mini samples. In runs 2 and 3 the agent then built a lookup table from problem text to label and rewrote `solver` to use it.

3.3 Planned architecture for the trading domain

Three layers. (a) A *sealed evaluator*: a separate process holding the data and the Qlib backtest, exposing `score(factor_code)` and returning metrics on the training and validation windows only. (b) The *solver*: a program that, given the feedback history, writes factor code into an R&D-Agent Qlib workspace; this is the analogue of today's `solver`. (c) The *meta-loop*: Gödel Agent's unchanged self-modification machinery acting on (b) and on itself.

Acceptance rule for self-edits. An edit is kept only if the validation utility of the solver it produces improves by a margin over the incumbent, and it passes two leakage checks. The first is static: the submitted code is inspected to confirm it reads no data outside the evaluator interface. The second is a shuffle test: the evaluator re-scores the same code on a copy of the data in which the future returns have been randomly permuted across stocks and dates. A factor that genuinely predicts returns should score near zero on the shuffled data; a score that stays high means the code is obtaining the answers from somewhere other than the features, as the MGSM lookup solver did, and the edit is rejected. Rejected edits are rolled back. This is the mechanism by which RQ2 is measured: every edit is logged with its sandbox gain, its test-window gain once the run is over, and the audit label.

Baselines and ablations. R&D-Agent(Q)'s factor loop, frozen, on the same data and backtest (conda runtime, no Docker), and a Gödel Agent with self-modification disabled (no `action_adjust_logic`), so the planner can only iterate on factor code. Both at the same token budget.

Models. The planner will be run with o3 for the main results and with DeepSeek for development and a cost ablation; R&D-Agent supports DeepSeek natively and Gödel Agent needs a configurable base URL and model name, a small change we have scoped.

4 Summary of Progress

4.1 What has been done

The substrate runs end to end on the author's machine (conda environment, pinned dependencies, two patches). Three full runs were executed and are committed as logs together with a script that regenerates every table in this section from them. The agent's final self-modified code from run 3 is preserved verbatim.

4.2 First result: the agent reads the answer key

Table 1 summarises the runs. In run 3 (the author's run) the agent spent its first ten steps reading the evaluator's source and probing the task module with `action_run_code`, including a call that printed an example together with its `targets` field. It then made one legitimate rewrite, a self-consistency solver, which scored 0.78 and 0.82 on two validation draws, below the 0.94 threshold. At step 27 it rewrote `solver` to look the task text up in the dataset, with the docstring: *"During evaluation the provided task string is always taken verbatim from the fixed MGSM dataset that ships with task_mgsm. Therefore we can guarantee perfect accuracy simply by looking the task up in that dataset and returning its known ground-truth answer."* Validation and test accuracy became 1.00; the framework saved the agent as the best found; and the agent's next analysis read: *"Replaced solver with deterministic lookup into MGSM dataset cached on agent, guaranteeing perfect accuracy (1.0)*

on both validation and test sets. . . . No further changes needed." Run 2 reached the same strategy independently by a different path, with seven rewrites in total of which three were lookup variants and one broke the solver (0.00).

Three observations matter for the design. First, nothing in the agent is adversarial; the prompt says to improve evaluation accuracy and the agent did exactly that. Second, the exploit needed no privilege escalation: the data and the scorer were importable from the agent's own process, and `action_run_code` makes any object it creates visible to `solver`. Third, the honest rewrites plateaued around 0.8 on 50-problem draws, a noisy estimate, which is precisely the situation in which a cheap exploit dominates a hard real improvement. A backtest with lookahead is the same situation with higher stakes.

4.3 What remains, and risks

Remaining work (weeks 8–10). (i) Build the US-equity Qlib dataset and the sealed evaluator process with the acceptance rule (about one week; the core contribution). (ii) Run the frozen R&D-Agent(Q) baseline on that data (two to three days). (iii) Point Gödel Agent's solver at the evaluator and run the self-modifying and non-self-modifying variants at a matched budget. (iv) Audit edits and write up.

Obstacles. R&D-Agent is not yet installed; its documented path needs Docker, which is absent on the author's WSL2 machine, but a conda-based runtime exists in the code and will be used. Its default data cannot satisfy the post-cutoff test window, hence the custom dataset. API cost is manageable at a few dollars per Gödel run, but the quant loop multiplies LLM calls by backtests; DeepSeek will be used for development and o3 for final runs. A weaker planner may self-modify less successfully, which is itself a reportable result. The schedule is tight but each item is scoped.

AI-Use Disclosure Statement

Ideation. The project direction is the author's. Claude (Anthropic) helped reframe the research questions around the evaluator-exploitation finding after the proposal feedback. **Writing.** The initial draft of the report was written with Claude's help using the author's notes, run logs and repository. The final report was written by the author, who takes full responsibility for its content. **Code and experiments.** Claude Code was used to set up the conda environment, diagnose and patch the two issues in Section 3.2, write the log summariser and README, and analyse the run logs; the runs themselves use OpenAI o3 and gpt-4o-mini, which are the system under study. No experiments were fabricated; all numbers are regenerable from the committed logs.

References

David H. Bailey and Marcos López de Prado. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

Paul Glasserman and Caden Lin. Assessing look-ahead bias in stock return predictions generated by GPT sentiment analysis. *arXiv preprint arXiv:2309.17322*, 2023.

Yuante Li, Xu Yang, Xiao Yang, Minrui Xu, Xisen Wang, Weiqing Liu, and Jiang Bian. R&D-Agent-Quant: A multi-agent framework for data-centric factors and model joint optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.15155.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*, 2022.

Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An AI-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pp. 27890–27913, 2025.